

Devtoberfest by
SAP TechEd



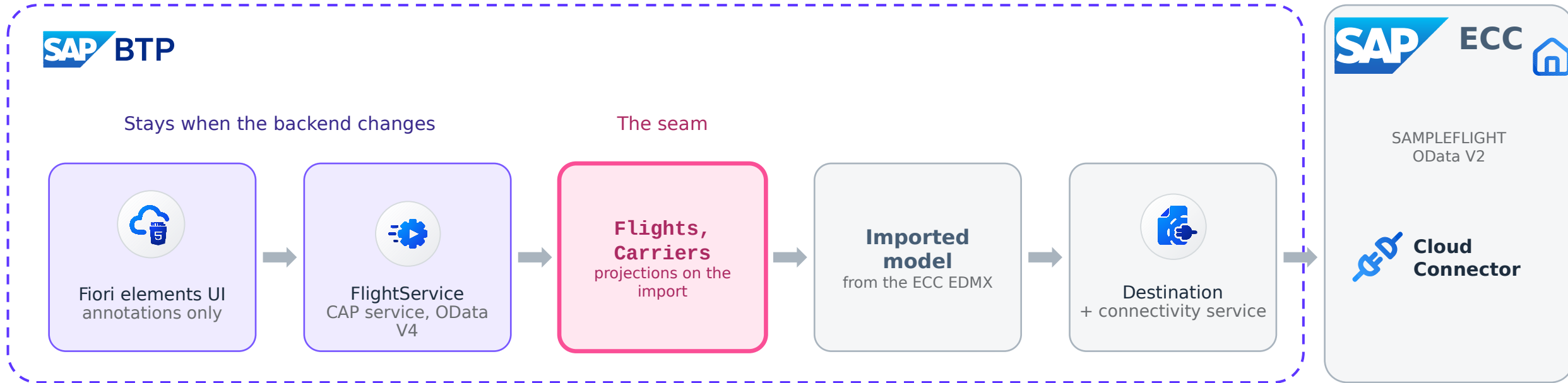
The ECC-to-CAP starter kit

You don't have to wait for S/4HANA
to start building on SAP BTP.

Sebastiano Marchesini, Avvale
Devtoberfest 2026

What we are building

CAP on SAP BTP, Cloud Foundry Environment, with no database: every read goes live to ECC's existing OData V2 service.



Before you start: what you need, and what this kit does not do

Prerequisites

- **ECC on AS ABAP 7.40+** with embedded Gateway (SAP_GWFND), no hub
- **SAMPLEFLIGHT active** in /IWFND/MAINT_SERVICE, ICF node on
- **SAP Cloud Connector** tunnelling ECC into your BTP subaccount
- **BTP subaccount** with Cloud Foundry, SAP Business Application Studio and a destination to ECC

Limitations

- **One technical user:** basic authentication, no principal propagation
- **No database, no cache:** every read goes live to ECC
- **Not covered today:** Cloud Connector setup

Walk the path first

01

ECC

[/IWFND/MAINT_SERVICE](#)

What to look for

- SAMPLEFLIGHT in the service catalog
- System alias LOCAL — an embedded Gateway, no hub needed
- Its ICF node is active

02

SAP Cloud Connector

[Connectivity > Cloud Connectors](#)

Not today's topic

It tunnels BTP into your network. Here it exposes ECC as ecc-virtual:8001, and only `/sap/opu/odata`.

Start here

- [SAP Help — SAP BTP Connectivity > Cloud Connector](#)
- [Discovery Center — Set up connectivity between SAP ERP and SAP BTP](#)

03

BTP destination

[Destinations > ecc-gateway](#)

The properties that matter

ProxyType: OnPremise

URL: `http://ecc-virtual:8001`

Authentication: BasicAuthentication

sap-client: `<your client>`

`WebIDEEEnabled: true`

`WebIDEUsage: odata_gen`

`HTML5.DynamicDestination: true`

Miss the last one and the BAS proxy answers a plain 500.

Tick only what the read path needs

Ticked in the CAP Project wizard

- Runtime: Node.js — the simplest remote-service consumption
- Database: none — every read passes through to ECC
- Deployment: Cloud Foundry, MTA
- XSUAA, Application Router, HTML5 Application Repository — the Fiori app needs a login and a home
- Connectivity and Destination services — the road to ECC

Left out, on purpose

- Kyma, multitenancy, CI/CD
- Portal, logging and notification services
- Messaging — there are no events

Everything you tick ends up in mta.yaml — and then it is yours to run.

Import the contract, fix one line

```
cds import SAMPLEFLIGHT.edmx --as cds
```

→ srv/external/SAMPLEFLIGHT.cds, plus a kind: odata-v2 entry in package.json

The one line that needed a human

```
@odata.Type : 'Edm.DateTime'  
fldate : Date not null; // was: DateTime
```

Delete the annotation, retype the field. Both live in the generated file: redo them after every re-import.



Five lines of CDS, one line of JS

```
using SAMPLEFLIGHT as ecc
  from './external/SAMPLEFLIGHT';

service FlightService {
  @readonly entity Flights
    as projection on ecc.FlightCollection;
  @readonly entity Carriers
    as projection on ecc.CarrierCollection;
}
```

srv/flight-service.cds — its handler is flight-service.js. Same base name, or it never loads.

```
this.on('READ',
  ['Flights', 'Carriers'],
  req => ecc.run(req.query))
That is the entire implementation.
```

The projection is the seam

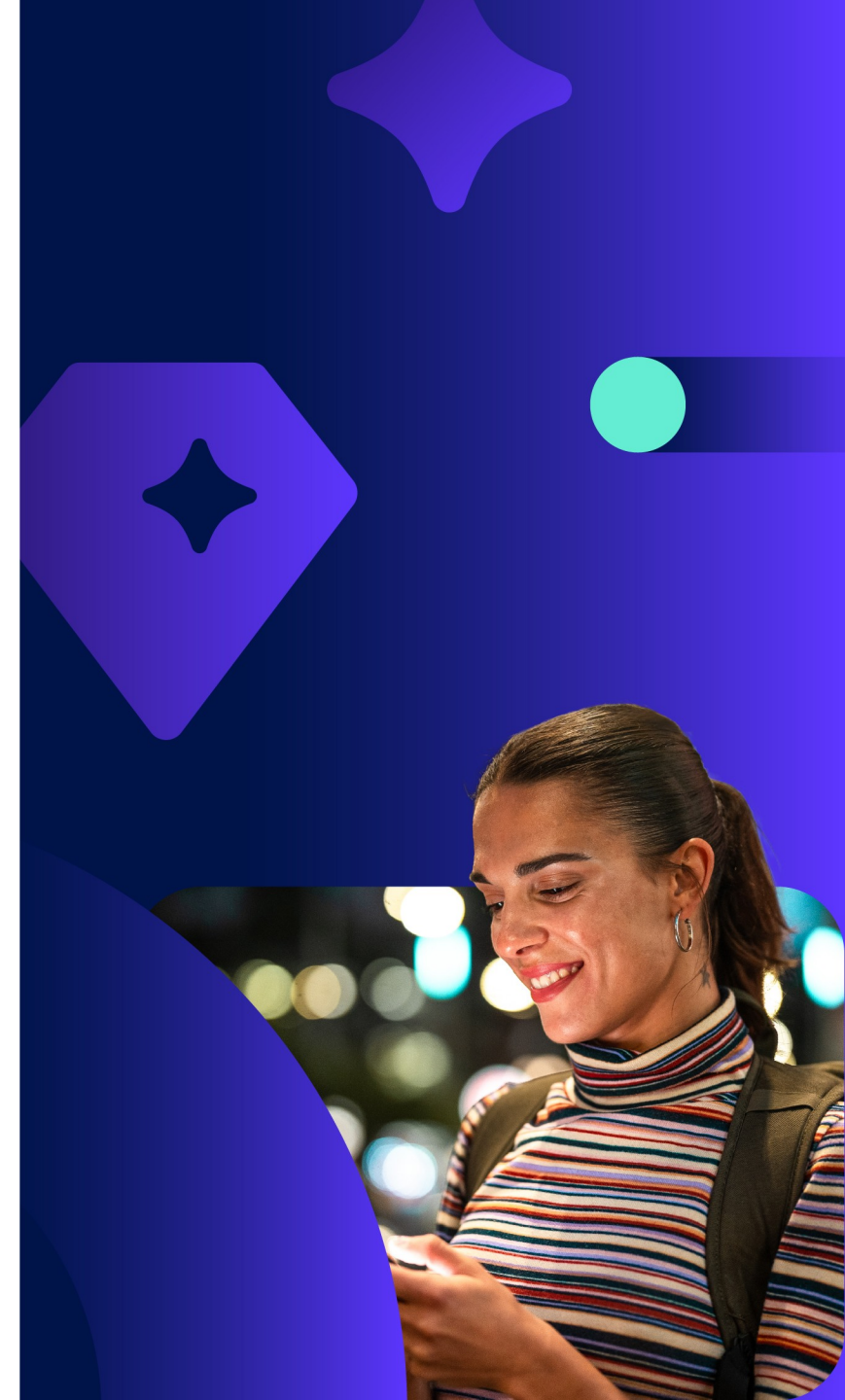
Your UI binds to FlightService, not to ECC's model. Today it passes ECC's names straight through.

It adds no database

Every read goes live to ECC. Filter and sort run in Gateway, not in CAP.

The runtime does the translation

CAP rewrites each OData V4 request into V2 for FlightCollection, and maps the result back.



The whole UI is annotations

```
annotate s.Flights with {
  carrid @title: 'Airline';
  connid @title: 'Flight';
  fldate @title: 'Date';
  price @title: 'Price' @Measures.ISOCurrency: currency;
};

annotate s.Flights with @(
  Capabilities.SearchRestrictions.Searchable: false,
  UI: {
    SelectionFields: [ carrid, connid, fldate ],
    ListItem: [ { Value: carrid }, { Value: connid },
                { Value: fldate }, { Value: price } ]
  }
);
```

app/flightproject/annotations.cds, trimmed — the full file adds plane, seats and the Object Page.

List Report and Object Page, no UI5 code

Fiori elements renders both from these annotations.

The labels are yours

ECC's metadata carries none. They live above the seam, which is where they belong.

Search is off, on purpose

ECC's service offers no search, so the UI doesn't either.

Filter and sort run in ECC

carrid eq 'AA' and \$orderby=fldate go straight through to Gateway.

Live demo: CAP serves ECC data as OData V4 and a Fiori app

Start CAP against live ECC

```
cds watch --profile bas
```

Run the Fiori app

Open localhost:4004 and click the flightproject web app link.
Press Go, filter Airline = AA, sort by date, open a flight.

Review the OData V4 API

Append to the BAS preview URL for port 4004:

```
/odata/v4/flight/Flights?$filter=carrid eq 'AA' . /odata/v4/flight/$metadata
```

CAP turns V4 into V2 for ECC; filter and sort run in ECC, not in CAP.

Deploy (show or mention)

```
mbt build  
cf deploy mta_archives/<name>.mtar
```

Then one line in the BAS terminal brings back ECC data, through BTP:

```
curl -s "http://ecc-gateway.dest/sap/opu/odata/IWFND/SAMPLEFLIGHT/$metadata" | head -c 800
```



Moving to S/4HANA changes the backend APIs, not your app

Usually stays put

- Your CAP service and the contract your UI consumes
- Fiori apps and annotations built on that service
- Business logic in your CAP handlers
- Tests written against your own service
- mta.yaml, the deployment and the destination pattern

Usually changes

- The APIs you call: custom SEGW services give way to released APIs or RAP services
- Entity and field names, so the projections start mapping them back
- Many more released, standard APIs to build on

What catches most projects out

- The protocol changes sometimes, not always: the Business Partner API is still OData V2, newer RAP services are V4.
- Same name, new meaning: a field can keep its label and change what it counts. Test the values, not just that the call succeeds.
- **After a system conversion, old custom services often still run. They are not clean core and cannot move to Public Cloud.**

Start Monday

Step 4 needs no ECC and no BTP account.

01 Activate the service, load the data

SAMPLEFLIGHT in /IWFND/MAINT_SERVICE — no ABAP to write.
Empty SFLIGHT tables? Run report SAPBC_DATA_GENERATOR first.

02 Create the project

In BAS: the CAP Project wizard, only the read path ticked.
Locally: `npm i -g @sap/cds-dk`

03 Clone the repo

The five files from this session, date fix included.

04 Run it with no backend

`cds watch` with no profile: CAP mocks SAMPLEFLIGHT in-process, with the sample rows in `srv/external/data`.

What your ECC actually needs

- AS ABAP 7.40 or higher
- Embedded Gateway via SAP_GWFND — a standalone Gateway hub is not required
- SAP Cloud Connector for the tunnel to your BTP subaccount
- The OData and ICF services active in the backend

SAP Note 3782867 · SAP Note 2217489

github.com/sebastianomarchesini/ecc-to-cap-starter-kit

Ask me anything in the Devtoberfest chat — I am in there all session.

Next: deploy with `mbt build` and `cf deploy`, then write-back and principal propagation.



Thank you.



© 2026 SAP SE or an SAP affiliate company. All rights reserved. See Legal Notice on www.sap.com/legal-notice for use terms, disclaimers, disclosures, or restrictions related to this material.

